Working with Navigation Bar

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}
```

Tells the Flutter framework to inflate the MyApp widget and make it the root of the widget tree displayed on the screen.

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: "Navigation Activity",
      theme: ThemeData(colorScheme: ColorScheme.light()),
      home: HomeScreen(),
    );
  }
}
```

```
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  // A simple screen: a big icon with a title
  Widget buildPlaceholderPage(IconData icon, String title) {
    return Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Icon(icon, size: 80),
          SizedBox(height: 16),
          Text(
            title,
            style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
          ),
        ],
      ),
    );
  }
```

```
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Navigation")),
      // Always shows the Home page for now
      body: buildPlaceholderPage(Icons.home, "Home"),
      bottomNavigationBar: BottomNavigationBar(
        type: BottomNavigationBarType.fixed,
        currentIndex: 0,
        selectedItemColor: Colors.orange,
        unselectedItemColor: Colors.black,
```

```
        onTap: (index) => print("Tapped index $index"),
        items: const [
          BottomNavigationBarItem(icon: Icon(Icons.home), label: "Home"),
          BottomNavigationBarItem(icon: Icon(Icons.star), label: "Favorite"),
          BottomNavigationBarItem(icon: Icon(Icons.settings), label: "Settings"),
          BottomNavigationBarItem(icon: Icon(Icons.share), label: "Share"),
        ],
      ),
    );
  }
}
```

- *buildPlaceholderPage(...): A helper method that creates a reusable layout template featuring a centered icon, vertical spacing (SizedBox), and bold text.*
- *body: buildPlaceholderPage(...): Hardcodes the body to always show the Home page regardless of user interaction.*
- *onTap: (index) => print("Tapped index $index"): In this static version, tapping a tab only prints a message to your debug console rather than changing the screen.*

Next we are going to make our navigation Work here through the use of:

1. **Stateful Widget**
2. **Helper Methods**
3. **Changing the Scaffold**

```
// Changed to StatefulWidget so the screen can change when a tab is tapped
class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  int _selectedIndex = 0; // which bottom bar tab is active
```

- ***StatefulWidget**: Used when a widget needs to change its visual state based on user interaction (like tapping a bottom navigation bar tab).*
- ***_selectedIndex**: A state variable that tracks which tab number is currently active (starts at 0 for Home).*

```
Widget buildPlaceholderPage(IconData icon, String title) {
    return Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Icon(icon, size: 80),
          SizedBox(height: 16),
          Text(
            title,
            style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
          ),
        ],
      ),
    );
  }

  // Pick which page to show based on the selected tab
```

```
  Widget buildPage() {
    if (_selectedIndex == 0) {
      return buildPlaceholderPage(Icons.home, "Home");
    } else if (_selectedIndex == 1) {
      return buildPlaceholderPage(Icons.star, "Favorites");
    } else if (_selectedIndex == 2) {
      return buildPlaceholderPage(Icons.settings, "Settings");
    } else {
      return buildPlaceholderPage(Icons.share, "Share");
    }
```

buildPage()*: Evaluates the current value of _selectedIndex using conditional statements (if/else) and returns the appropriate placeholder page (Home, Favorites, Settings, or Share) dynamically.*

```
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Navigation")),
      body: buildPage(),
      bottomNavigationBar: BottomNavigationBar(
        type: BottomNavigationBarType.fixed,
        currentIndex: _selectedIndex,
        selectedItemColor: Colors.orange,
        unselectedItemColor: Colors.black,
        onTap: (index) {
          setState(() {
            _selectedIndex = index;
          });
        },
        items: const [
          BottomNavigationBarItem(icon: Icon(Icons.home), label: "Home"),
          BottomNavigationBarItem(icon: Icon(Icons.star), label: "Favorite"),
          BottomNavigationBarItem(icon: Icon(Icons.settings), label: "Settings"),
          BottomNavigationBarItem(icon: Icon(Icons.share), label: "Share"),
        ],
      ),
    );
  }
}
```

- ***Scaffold****: Provides the core high-level structure for the screen, providing slots for an appBar, body, and bottomNavigationBar.*
- ***currentIndex****: _selectedIndex: Highlights the correct tab icon in the bottom navigation bar depending on the current state.*
- ***setState(() { _selectedIndex = index; })****: Crucial Flutter mechanism. When a tab is tapped, setState updates _selectedIndex and tells Flutter to rebuild the UI so buildPage() displays the newly selected tab's content immediately.*
- ***BottomNavigationBarItem****: Defines each individual tab's icon and text label displayed at the bottom of the screen.*